

Banking Loan Risk Analysis Application

Overview

Banks and financial institutions process large volumes of customer and credit information to monitor repayment behavior, account exposure, and credit risk. Analysts need reliable systems for importing customer data, validating records, calculating financial indicators, and generating risk reports.

Manual spreadsheet-based processing can make loan-risk analysis slow and difficult to maintain.

Banking teams often ask questions such as:

- Which customer groups have higher payment-risk indicators?
- How do credit limits, payment history, bills, and previous payments vary across customers?
- Which records require additional review based on defined business rules?
- What are the monthly and segment-level trends in credit exposure and payment behavior?

Build a Python-based full-stack banking analysis application that performs ETL, stores curated data in a relational database, exposes APIs, and provides dashboards for credit-risk analysis using transparent business rules and analytics.

Questions

- Which customer segments have higher payment-risk indicators?
- How do credit limits and payment behavior vary across segments?
- Which records require additional review using defined business rules?
- What are the trends in credit exposure, bills, and previous payments?

Requirement 1 – Basic

- Build a Python ETL pipeline for customer credit data
- Clean and transform demographic, credit-limit, repayment, bill, and payment fields
- Design relational tables for customers, credit profiles, bills, and payments
- Load curated data into PostgreSQL/MySQL
- Create SQL queries for exposure, repayment, bill, payment, and segment KPIs
- Implement REST APIs and CRUD operations using Flask or FastAPI
- Build a dashboard for credit exposure and payment-risk indicators
- Implement configurable business-rule validation and exception handling

Requirement 2 – Advanced

- Implement incremental ETL and record-level reconciliation
- Create reusable data-quality checks for duplicates, invalid ranges, and inconsistent records
- Implement rule-based risk bands such as Low, Medium, and High using documented thresholds
- Add API pagination, filtering, sorting, and export endpoints
- Implement role-based access for analyst and administrator users
- Optimize database queries with indexes and aggregated reporting tables
- Add automated tests for ETL, business rules, SQL queries, and APIs

- Implement logging and scheduled batch processing

Build a simple front-end interface.

Deliverables

1. Architecture Diagram

- Include a high-level view of the system in JPEG or PDF.
- Show how data is processed from the provided dataset through ETL, database, APIs, and the user's interface.

2. Design

- Articulate system design decisions taken and the trade-offs made.

3. Full Executable Code (Microservice)

- Focus on clarity and modularity.
- Provide a README explaining:
 - Project setup (how to install or run locally)
 - Sample usage (for example, a test API request and resulting output)
 - Database setup and schema
 - ETL execution steps

4. Panel Presentation (10 minutes)

- Demo of the working solution outlining the design and approach (8 minutes).
- Q&A (2 minutes).

Dataset Name

Default of Credit Card Clients

Primary Link

UCI Machine Learning Repository – Default of Credit Card Clients

<https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>

Alternative Link

UCI Default of Credit Card Clients – dataset documentation

<https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>

Format

XLS / CSV

Key Fields

ID, LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE, PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6, BILL_AMT1-BILL_AMT6, PAY_AMT1-PAY_AMT6, default.payment.next.month

Reason

The UCI dataset contains 30,000 customer records with credit limits, demographic fields, monthly repayment status, bill amounts, previous payment amounts, and a default indicator. These variables support ETL, SQL analysis, business-rule risk bands, dashboards, APIs, and data-quality exercises.